DALIL TRAINING ACADEMY · FULL STACK WEB DEVELOPMENT

Expense Tracker Project

Student Roadmap: from Backend to Full Stack in 11 days

Duration	Work style	Phases	Delivery
11 days	**Individual**	**4 phases**	**Zip + Demo**

About the project

You will build a web app to track your personal expenses. You add an expense (title, amount, category, date), see all your expenses in a table, filter them by category, and see the total, the number of expenses, and the highest one right away. The data is saved in PostgreSQL through a backend built with Node.js and Express, and the front-end talks to it through an API.

What you already know, and what is new

- **Already covered:** HTML, CSS, JavaScript (arrays, objects, DOM, events), Bootstrap, and PostgreSQL queries (SELECT, INSERT, UPDATE, DELETE). This is enough to build the whole backend and front-end.
- **New:** JSON, Promises, async/await and fetch, your first steps with Node.js and Express, and connecting Node to PostgreSQL. Nobody will teach these in a lecture. You will learn them on your own in Phase 0.

What is the goal?

To apply what you learned in a complete project, to learn new tools on your own, and to understand how the front-end talks to the backend and the database. It also helps us see where you are now, and where we should focus with you in the next part of the program.

How we work

- The project has 4 phases (Phase 0 to Phase 3). Each phase has a goal, requirements, and a checklist called “I am done when…”.
- **This time the backend is built before the front-end.** After Phase 0 you build the whole API, and only then do you build the interface, connected to it from day one.
- Do not move to a new phase before you finish the checklist of the previous one.
- There is **one official Checkpoint**: the end of Day 9, when the whole project works. The final delivery is on Day 11.
- Day 10 is flexible: if you are behind, you catch up. If you finished early, you start the Bonus.
- **Stuck on the backend?** Ask in Discord with the same 30-minute rule. There is no fallback path, so keep going and ask for help instead of stopping.

Timeline

Day	Phase	Work	Delivery
1	Phase 0	Set up your environment. Read about JSON, fetch/async, and the backend/REST. Do task 1.	—
2	Phase 0	Read about Node/Express and Node with PostgreSQL. Do tasks 2 and 3.	Phase 0 tasks
3	Phase 1	Create the database and the `.env` file, connect Node to PostgreSQL, and build the first endpoints (GET and POST).	—
4	Phase 1	The other endpoints (PUT and DELETE) and validation.	—
5	Phase 1	Test every endpoint with Thunder Client.	API test screenshots
6	Phase 2	Page structure with Bootstrap, and fetch the data straight into the table.	—
7	Phase 2	The add form with validation connected to POST, and delete connected to DELETE.	—
8	Phase 2	The filter, and editing in a modal connected to PUT.	—
9	Phase 2	Spinner, alerts, and try/catch for every case.	**Checkpoint**
10	Buffer	Catch up and finish the connection.	—
11	Phase 3	Responsive design, CSS Grid, Bonus, README, delivery, and demo.	**Final delivery**

The days are estimates. If you finish a phase early, continue with the next one.

Set up your environment (first hour of Day 1)

- **Node.js** (LTS version) from the official website nodejs.org.
- **VS Code** with the extension called Live Server, to run the front-end.
- **Thunder Client**: an extension inside VS Code to test the API. Search for it in the Extensions tab and install it.
- **PostgreSQL** with pgAdmin (both installed for the database course). Make sure PostgreSQL is running, and that you can open pgAdmin and connect with your password.

To check that Node works, open the terminal and type:

```
node -v
npm -v
```

You should see a version number for each command. If you get an error, take a screenshot and write in Discord.

Project structure and data shape

You will receive the structure ready (the starter), with a `schema.sql` file that has the expenses table and some sample data. The frontend and the backend are in two separate folders because they are really two applications that talk to each other:

```
expense-tracker/
├── frontend/
│   ├── index.html
│   ├── css/style.css
│   └── js/app.js
├── backend/
│   ├── server.js
│   ├── package.json
│   ├── schema.sql
│   └── .env.example
└── README.md
```

The shape of an expense (JSON)

```
{
  "id": 1,
  "title": "Lunch",
  "amount": 4.5,
  "category": "Food",
  "date": "2026-01-15"
}
```

Allowed categories: `Food`, `Transport`, `Bills`, `Entertainment`, and `Other`. In the database, an expense is a row in the `expenses` table with the same columns. The `id` is created by the database.

The phases in detail

Phase 0: Self-study — Days 1 – 2

Goal: learn the new tools before you need them. You will get a separate file with selected resources for each topic, so you do not waste time searching.

Topics

- **JSON:** what it is, `JSON.stringify`, and `JSON.parse`, and why it is the format the front-end and backend use to talk to each other.
- **Promises, async/await, and fetch:** GET and POST requests.
- **The backend and REST:** what a request and a response are, the basic status codes (200, 201, 400, 404, and 500), and the verbs GET, POST, PUT, and DELETE.
- **Node.js, npm, and Express:** your first route that returns JSON, and why we need CORS.
- **Node.js and PostgreSQL:** the `pg` library, sending queries with parameters (`$1` and `$2`), and the `.env` file.

3 small tasks to check that you understood

Task	What to do	How you know it is done
Task 1	Get data from a public API with `fetch` and show it in Bootstrap cards.	The cards appear on the page from real data.
Task 2	Build an Express server with a route that returns JSON.	You see the JSON from the browser and from Thunder Client.
Task 3	Connect Express to PostgreSQL: a route that reads a practice table with `pg` and with parameters.	You see the table data in Thunder Client, and a wrong `id` does not crash the server.

Checklist: I am done when…

- ☐ I solved the three small tasks and all of them work.
- ☐ I can explain in my own words the difference between GET and POST, and what status `404` means.
- ☐ I know why the front-end needs CORS to talk to the server.
- ☐ I know why we send values as parameters (`$1`) instead of joining them into the SQL text.
- ☐ **I submitted** a folder with the three tasks (Day 2).

Phase 1: Build the whole Backend — Days 3 – 5

Goal: build a complete API that manages the expenses and stores them in PostgreSQL, and test it fully on its own before you write a single line of HTML.

Prepare the database (first hour of Day 3)

- Create an empty database named `expense_tracker` in pgAdmin.
- Run the `schema.sql` file on it. It creates the `expenses` table and adds some sample data.

- Copy `.env.example` to a new file named `.env` and write your PostgreSQL password in it. **The `.env` file is never uploaded and never submitted.**
- Install the packages: `npm install express cors pg dotenv`.

Required endpoints

Method	Path	What it does	Success	Errors
GET	/api/expenses	Returns all expenses	200	–
GET	/api/expenses/:id	Returns one expense	200	404
POST	/api/expenses	Adds a new expense	201	400
PUT	/api/expenses/:id	Updates an expense	200	400, 404
DELETE	/api/expenses/:id	Deletes an expense	200	404

How to try an endpoint with Thunder Client

- **Start the server** from the terminal and keep it running.
- **Open Thunder Client** from its icon in the VS Code sidebar and click **New Request**.
- **Choose the method** (GET or POST…) and write the URL, for example `http://localhost:3000/api/expenses`.
- **For POST and PUT:** open the **Body** tab, choose **JSON**, and write the expense data.
- **Click Send.** You will see the status code and the JSON that came back next to the request.

Work rules

- **Every query uses parameters:** `pool.query("... WHERE id = $1", [id])`. Never join user data into the SQL text, because this causes SQL injection.
- The database creates the `id`. Use `RETURNING` to send back the new row.
- Be careful: the `pg` library returns `NUMERIC` as text and `DATE` as a Date (and the day can shift). Fix both in the SELECT (there is a hint in the starter).
- Validation in the server: if the data is missing or wrong, return `400` with a JSON that has a `message` explaining the problem. The category must be one of the five allowed values, and the amount must be a number greater than 0. Check the `id` before the query, and if it does not exist or is not valid, return `404`.
- Enable CORS so the front-end can talk to the server.
- Test every endpoint with Thunder Client **before** you start Phase 2 (the front-end).

Checklist: I am done when…

- ☐ All 5 endpoints work and return the right status code.
- ☐ If I send wrong data, I get `400` with a clear message.
- ☐ After I stop the server and start it again, the data is still there (in the database).
- ☐ Every query uses `$1` and `$2`, and no SQL text is joined with user data.
- ☐ The date comes back in the format `YYYY-MM-DD`.
- ☐ **I submitted** a screenshot of each endpoint test (success and error) with the `backend` folder (Day 5), without the `.env` file.

Phase 2: Build the Front-end, connected to the API — Days 6 – 9

Goal: build the whole interface, with every feature working through `fetch` straight against the API you built in Phase 1. **There is no in-memory version this time**: from day one the server must be running for you to try your work.

What to build

- **Navbar** and **3 summary cards**: total amount, number of expenses, and highest expense. The cards count **all** the expenses, not the filtered list.
- **Form** to add an expense: title (required), amount (a number greater than zero), category (select), and date. It needs validation with clear error messages, and sends the data with `POST` on success.
- **Table** with these columns: title, amount, category (a coloured badge), date, an edit button, and a delete button.
- **Filter** by category, with an “All” option.
- **Edit an expense** in a Bootstrap modal filled with the current data. Saving sends `PUT`.
- **Delete an expense** with `DELETE`.

Work rules

- Every operation (list, add, edit, delete) uses `fetch` with `async/await` from the moment you write it. Use `try/catch`.
- After every add, edit, or delete, ask the server for the list again (`GET`) so the page always shows the truth.
- Show a Bootstrap **spinner** while the data loads.
- Show every error in a clear Bootstrap **alert**. If the server is off, the message must make sense to the user. If it returns `400`, show the `message` it sent.
- Do not use any JavaScript library except Bootstrap.

Checklist: I am done when…

- ☐ I can add an expense and it appears in the table (after the server confirms it was saved).
- ☐ The form does not accept an empty field, or an amount of zero or less.
- ☐ I can delete an expense: it disappears from the table and the summary updates.
- ☐ The category filter works, and I can go back to “All”.
- ☐ I can edit an expense from the modal, and the table and summary update.
- ☐ The spinner appears while loading, and disappears after.
- ☐ When I stop the server, a clear alert appears, not an empty or broken page.
- ☐ The page looks good on a phone screen.

Checkpoint: end of Day 9
Submit the whole project working (front-end + backend + database). This is the first and most important official evaluation point of the project. Day 10 is flexible for anyone who is not finished yet.

Phase 3: Polish and Delivery — Day 11

Goal: make the project ready for delivery, and add your own touch.

Required

- **Responsive design:** test it on a phone screen width in DevTools.
- **CSS Grid:** use it to show the summary cards (a new topic, so search for it).
- **Clean code:** clear names, comments where needed, and no leftover `console.log`.
- **README:** how to run the project from zero (create the database and run `schema.sql`, write `.env`, install the packages, start the server, and open the front-end), screenshots of the app, and a short section about the hardest thing you faced and how you solved it.
- **Clean delivery:** without the `node_modules` folder and without the `.env` file.

Bonus (choose one or more)

- A chart of the expenses by category (Chart.js).
- Filter by month, or search by title.
- Sort the table by clicking a column title.
- Export the expenses as a CSV file.
- Dark mode.

Checklist: I am done when…

- ☐ The project works from the first try by following the README steps.
- ☐ The page looks good on both phone and desktop.
- ☐ I can explain any line in the project.

How to ask for help

- **The 30-minute rule:** if you are stuck for more than 30 minutes on one point, ask in Discord. There is no need to lose a whole day on something small.
- **Since the front-end is connected to the backend from day one this time**, a backend problem can stop you completely. Do not hesitate to ask early.
- **Before you ask:** open the Console (F12), read the error message, and try `console.log` to see what is happening.
- You may search and use outside resources, but **you must be able to explain every line you submit.**

How to write a good question

What	Why
What do you want to do?	So we understand the goal, not only the error.
What code did you write?	A screenshot or text of the related part only, not the whole project.
What happened?	The exact error message, or a screenshot.
What did you try?	So we do not suggest something you already tried.

Delivery and evaluation

What do you submit?

- **End of Phase 0 (Day 2):** a folder with the three tasks.
- **End of Phase 1 (Day 5):** the `backend` folder with screenshots of every endpoint test.
- **Checkpoint (end of Day 9):** the whole project works (front-end + backend + database).
- **Final delivery (Day 11):** the whole project folder as a zip file, **without** the `node_modules` folder and **without** the `.env` file, with a README. After that, a short demo of the project.

What do we evaluate?

- **Functionality:** every required feature works and meets the checklist.
- **Code quality:** organization, clear names, and no repeated or leftover code.
- **Following the requirements:** the structure, the backend code, and the way of validation.

- **Self-learning:** Phase 0, how you handled the new tools, and the Bonus.
- **The Demo:** how well you understand the code you wrote. This is the main way we measure your front-end fundamentals in this project.

Good luck!
This project is your chance to see how much you can build on your own. Ask, try, and do not be afraid of mistakes. They are part of the work.